

# Sampling Based Planning

Robots in Context

Leon Bodenhagen [lebo@mmmi.sdu.dk](mailto:lebo@mmmi.sdu.dk)

Associate Professor

SDU Robotics, Maersk Mc-Kinney Moller Institute



# Agenda

- Presentations for PE4 (GVD + Brushfire)
  - August
  - Lukas
- Programming Exercise 5, volunteers:
  1. Jacob
  - 2.
- Sampling-based planning – lecture 6
- PE 6.1: RRT\*
- PE 6.2: PRM
- Midterm evaluation



# Content

- Planning in higher dimensional spaces
- Brief History
- Probabilistic Roadmaps
- Expansive-Space Trees
- Rapidly Exploring Random Trees
- Sampling Based Roadmap of Trees
- Path Post Processing



# Path Planning in Higher Dimensional Spaces

- Ideal: Create a complete or resolution complete planner
- Problem: "Curse of Dimensionality"
  - Time and memory increases exponentially with the dimensionality of the configuration space.
  - In general considered to be PSPACE-complete
- Need algorithms with a trade-off between completeness and practical efficiency.



# Monte Carlo Techniques

- Key Idea:
  - Instead of exhaustively exploring all possibilities, randomly explore a subset and keep track of progress.
  - Can quickly find solutions for easy cases.
  - Need to sacrifice completeness and optimality
- Gives rise to the probabilistic completeness property.

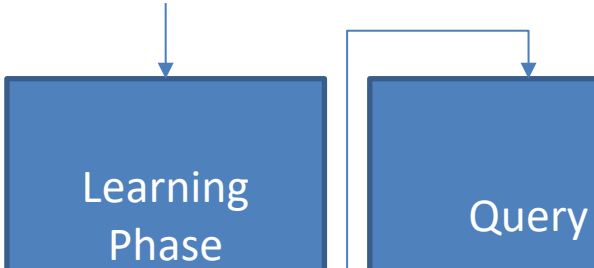


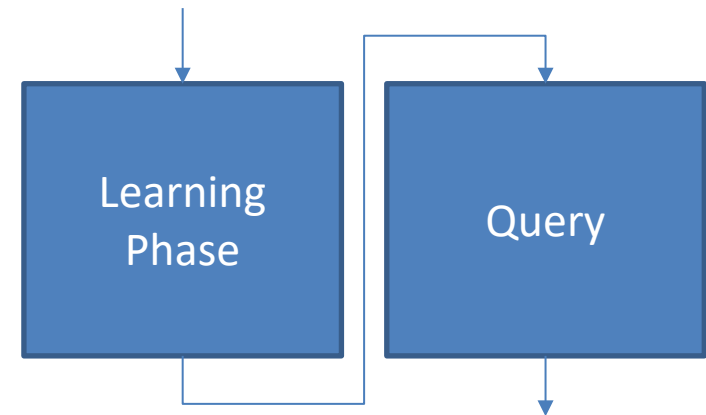
# Very Brief History

- Some of the first work using random sampling for planning in the early 1990'ies.
- Probabilistic Roadmaps: [1996 and forward]
  - Initially uniform sampling
  - Many papers since then published with alternative sampling strategies.
- Rapidly Exploring Random Trees [1998 and forward]
  - Many adaptations since.
- Prior to the sampling based planners potential field based have been popular.



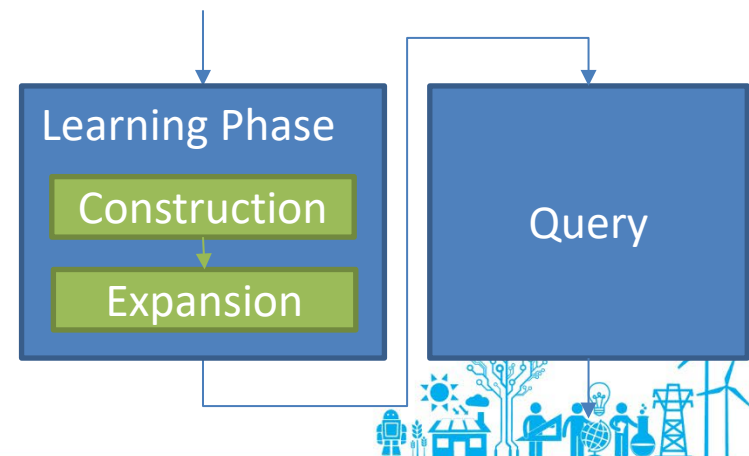
# Probabilistic Roadmaps (PRM)

- Original paper by Kavraki, Latombe, Overmars in 1996
  - Two Phases
    - Learning Phase - Offline
    - Query Phase – Online
  - Multi Query Planner
- 
- ```
graph TD; A[ ] --> B[Learning Phase]; B --> C[Query];
```
- The diagram illustrates the two phases of the Multi Query Planner. It consists of two blue rectangular boxes. The first box is labeled "Learning Phase" and has a blue arrow pointing down to it from above. The second box is labeled "Query" and has a blue arrow pointing down to it from above. A blue line connects the top of the "Learning Phase" box to the top of the "Query" box, indicating a flow or relationship between the two phases.

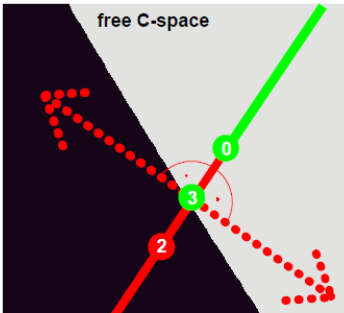


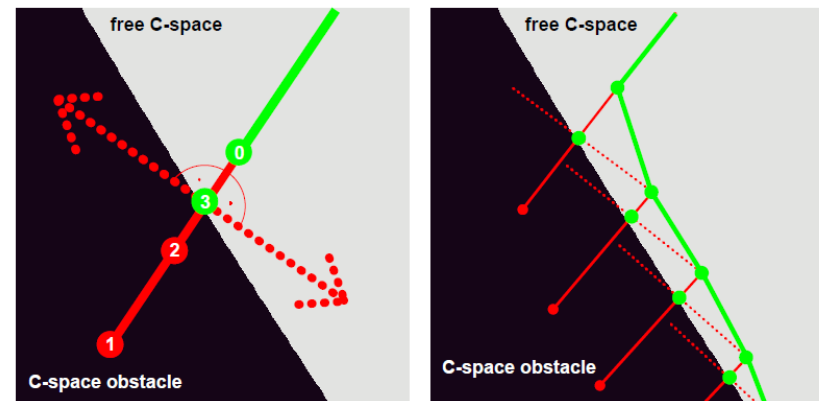
# PRM Learning Phase

- Construction Step
  - Construct a roadmap by generating random collision free configurations and connecting these with a simple, fast *local planner*.
  - Store the configurations and connections as a graph.
- (Expansion Step)
  - (Improve the connectivity)



# Local Planner

- Local Planner  $\Delta(q1, q2)$ 
    - Checks for a path between  $q1$  and  $q2$
    - Deterministic
    - Trade-off between fast and powerful
  - Examples
    - Straight line interpolation between  $q1$  and  $q2$ 
      - Check the line for collision
    - Follow the geometry
      - Baginski, The  $Z^3$  Method for Fast Path Planning in Dynamic Environments
- 



# Local Planner

- Baginski, The  $Z^3$  Method for Path Planning in Dynamic Environments, Proceedings of the 1995 IEEE Conf. Applications of Control and Robotics, Orlando, FL, pp. 47-52.

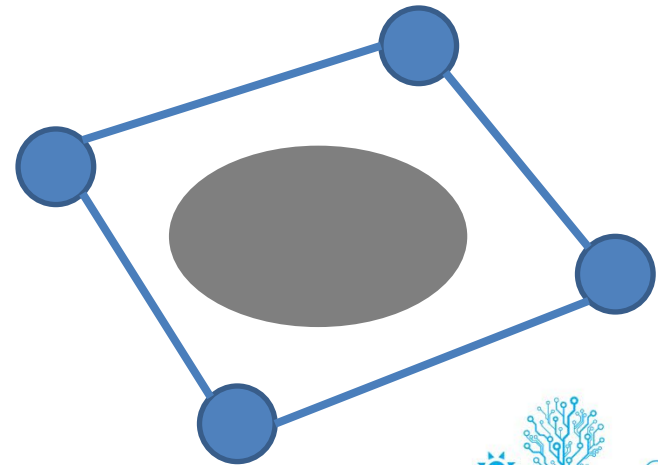


## Expansion

-

# Graph Properties

- The resulting graph might have cycles. Is that an advantage or a problem?
- Should the graph be directed or undirected?



# Expansion Step

Learning Phase

Construction



Expansion

- Find a node in a difficult region and expand it by adding nodes around it.
- What is a difficult region?



# Expansion Step

- Find a node in a difficult region and expand it by adding nodes around it.
- What is a difficult region?
  - Depending on the number of nodes close to  $q$
  - Distance to nearest connected component not containing  $q$
  - Depending on how often the local planner fails.
- Select node  $q$  s.t. the probability of selecting  $q$  is

$$P(q) = w(q) / \sum w(q_i)$$

where  $w(q)$  could be based on one of the criteria above



# Expansion Step

Expand( $q$ )

1.  $q' = q$ ;
2. **Repeat**
3.      $\text{dir} = \text{random direction (normalized)}$
4.     **While**  $(q' + \epsilon * \text{dir}) \in C_{\text{free}}$
5.          $q' += \epsilon * \text{dir}$
6.     **Until**  $\text{cnt}++ = S$
7.      $V = V \cup \{q'\}$
8.      $E = E \cup \Delta(q, q')$
9.     Connect to other components



# Time usage

- Time usage
  - $T_{\text{learning}} = T_{\text{construction}} + T_{\text{expansion}}$
  - $T_{\text{construction}} = 2/3 T_{\text{learning}}$
  - $T_{\text{expansion}} = 1/3 T_{\text{learning}}$
- Shows best performance on average.



# Using the PRM

- The PRM is used in a specific query to find a solution
- Input,  $q_{init}$  and  $q_{goal}$  which are both collision free
- Properties
  - Accessibility
  - Departability
  - Connectivity



# Query (Searching for a solution to a specific problem)

Input( $E, V, q_{\text{init}}, q_{\text{goal}}$ )

1.  $q_{init}'$  = closest neighbor for which  $\Delta(q_{init}, q_{init}')$  is collision free
2.  $q_{goal}'$  = closest neighbor for which  $\Delta(q_{goal}, q_{goal}')$  is collision free
3.  $V = V \cup \{q_{init}\} \cup \{q_{goal}\}$
4.  $E = E \cup \{(q_{init}, q_{init}')\} \cup \{(q_{goal}, q_{goal}')\}$
5.  $P = \text{shortestPath}(q_{init}, q_{goal})$



# Query (Searching for a solution to a specific problem)

Input( $E, V, q_{\text{init}}, q_{\text{goal}}$ )

1.  $q_{init}'$  = closest neighbor for which  $\Delta(q_{init}, q_{init}')$  is collision free

2.  $q_{gc}$  What if we cannot connect to the nearest neighbor?  
CO

3.  $V = V \cup \{q_{\text{init}}\} \cup \{q_{\text{goal}}\}$

- $$4. \quad E = E \cup \{(q_{\text{init}}, q_{\text{init}}')\} \cup \{(q_{\text{goal}}, q_{\text{goal}}')\}$$

5.  $P = \text{shortestPath}(q_{\text{init}}, q_{\text{goal}})$



# Distance Function

- Which distance function to use?



# Distance Function

- Which distance function to use?
- Ideally it should reflect the probability of a connection being accepted.
  - E.g.: for rigid body motion, translation and rotations could be considered separately



# Pros and Cons with PRM

## Pros

- Shorter run time
- flexibility

## Cons

- Solution not optimal
- Context specific Tuning required



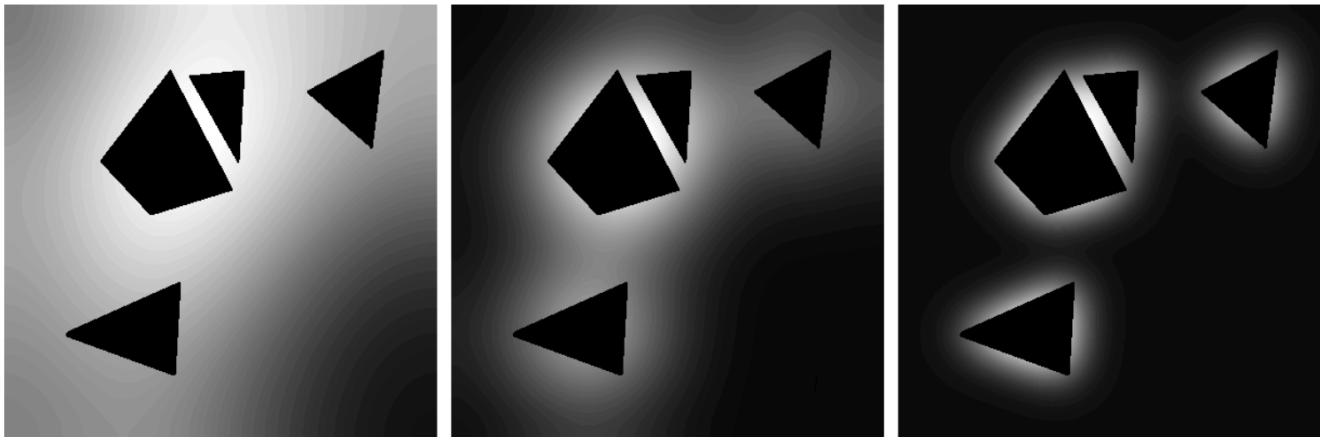
# Sampling Strategies

- **Uniform Random**
- Sample near obstacles OBPRM.
  - **Gaussian sampler**
- Sampling inside narrow passages
  - Bridge sampler
- Visibility Based
- Manipulability-Based Sampling
- **Quasirandom Samling**



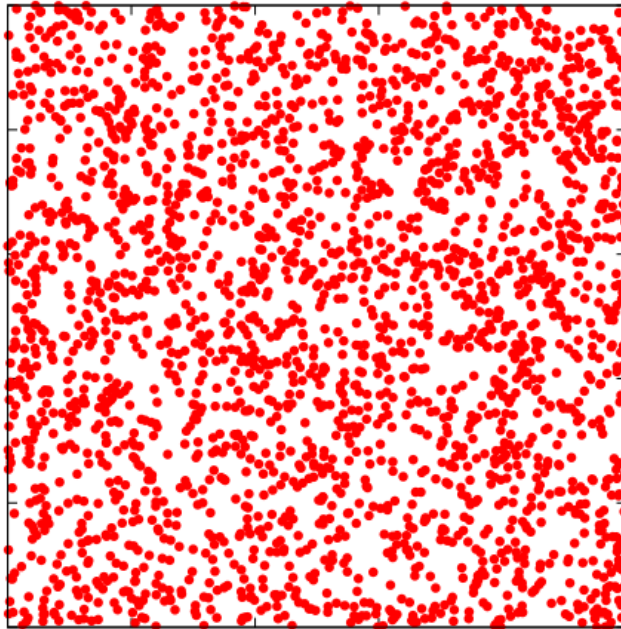
# Gaussian Sampling

- Generate random configuration  $q_1$
- Generate random configuration  $q_2$  close to  $q_1$
- If both  $q_1$  and  $q_2$  are in collision or both collision free discard both. Otherwise use the collision free configuration

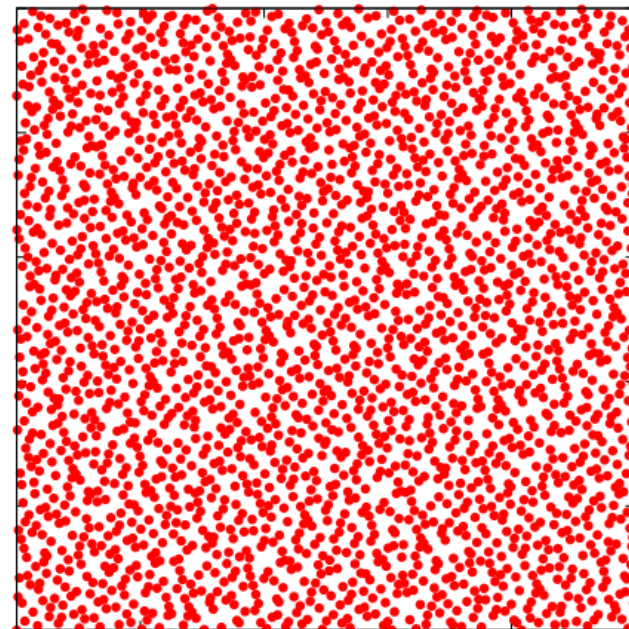


# Quasi Random Samling

- Uniformly Random Samling does not look all that uniform



Uniform Random



Quasi Random



# Single Query Planners

- We know the start and goal and wish to find a path as quickly as possible.
- We do not take into account potential later reuse of data.
- Two Examples:
  - Expansive Spaces Trees (EST)
  - Rapidly Exploring Random Trees (RRT)



# Expansive-Spaces Trees

## **buildEST(qstart, n)**

$V = \{q_{\text{start}}\}, E = \emptyset, T = \{V, E\}$

For ( $i = 1$  to  $n$  do)

$q$  = random configuration from  $V$  with probability  $\pi_T(q)$

extendEST( $T, q$ )

## **extendEST( $T, q$ )**

$q_{\text{new}}$  = random collision free configuration close to  $q$

if ( $\Delta(q, q_{\text{new}})$ )

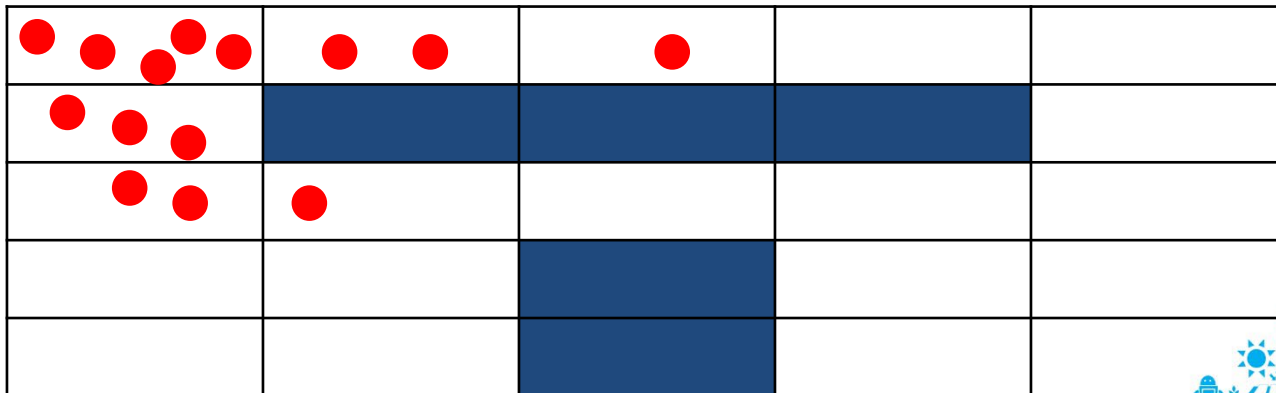
$V = V \cup \{q_{\text{new}}\}$

$E = E \cup \{(q, q_{\text{new}})\}$



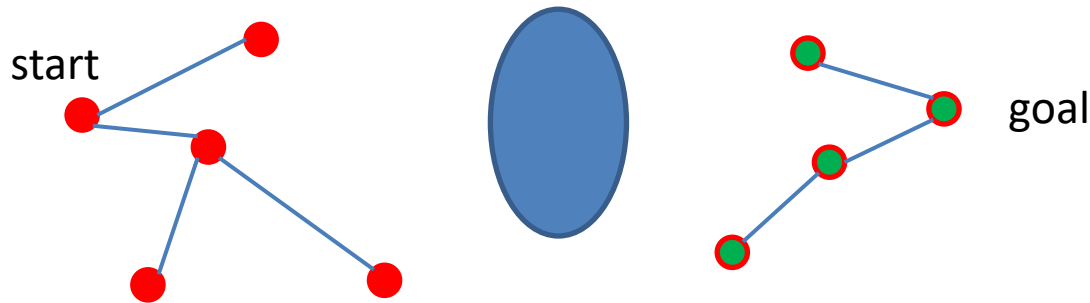
$$\pi_T(q)$$

- We wish to explore the areas with the least nodes.
- Ideally we could compute a density by the distance to other nodes.
- Put nodes into a grid.
  - Select a cell at random
  - Select a nodes at random within the cell



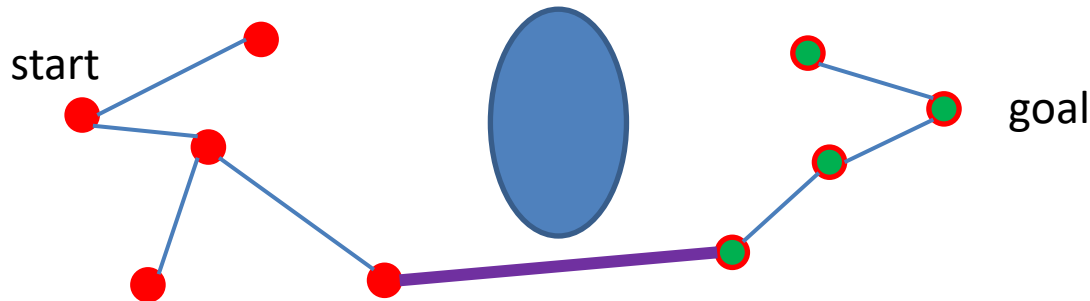
# Using EST as a Planner

- Create two trees (one from start and one from goal)
- In each step see if we can connect the newest in one tree to the closest in the other tree.



# Using EST as a Planner

- Create two trees (one from start and one from goal)
- In each step see if we can connect the newest in one tree to the closest in the other tree.



# Rapidly Exploring Random Trees (RRT)

## buildRRT(qstart, n)

$$V=\{q_{\text{start}}\}, E=\emptyset, T=\{V,E\}$$

For (i = 1 to n do)

q = random chosen collision free configuration

extendRRT( $T, q$ )

## extendRRT(T, q)

$$q_{\text{near}} = \text{closest neighbor of } q \text{ in } T$$

$q_{\text{new}}$  = take a step from  $q_{\text{near}}$  towards  $q$

if ( $q_{\text{new}}$  is collision free)

$$V = V \cup \{q_{\text{new}}\}$$
$$E = E \cup \{(q_{\text{near}}, q_{\text{new}})\}$$

```
return qnew
```

```
return NULL
```



# RRT Connect

An alternative to the previous method is to try forcing the trees together by repeating the extendRRT until collision or connection.

```
connectRRT(T, q)
  repeat
     $q_{\text{new}} = \text{extendRRT}(T, q)$ 
  until ( $q_{\text{new}} == q$  or  $q_{\text{new}}$ )
  if ( $q_{\text{new}} == q$ )
    return connected
  else
    return failure
```





$T_1$ : Tree with  $q_{\text{start}}$  as root

For (i = 1 to max attempts)

$q_{\text{rand}}$  = random collision free configuration

$$q_{\text{new},1} = \text{extendRRT}(T_1, q_{\text{rand}})$$

```
if (qnew,1 != NULL)
```

$$q_{\text{new},2} = \text{extendRRT}(T_2, q_{\text{new},1})$$

```
if (qnew,1 == qnew,2)
```

```
return merged
```

$$\text{swap}(T_1, T_2)$$

## Return failure



# RRT-Connect

## MergeRRT( $q_{\text{start}}$ , $q_{\text{goal}}$ )

$T_1$ : Tree with  $q_{\text{start}}$  as root

$T_2$ : Tree with  $q_{\text{goal}}$  as root

For ( $i = 1$  to max attempts)

$q_{\text{rand}}$  = random collision free configuration

$q_{\text{new},1} = \text{extendRRT}(T_1, q_{\text{rand}})$

if ( $q_{\text{new},1} \neq \text{NULL}$ )

$q_{\text{new},2} = \text{extendRRT}(T_2, q_{\text{new},1})$

if ( $q_{\text{new},1} == q_{\text{new},2}$ )

return merged



if( $\text{connectRRT}(T_2, q_{\text{new},1})$ )

return path

swap( $T_1, T_2$ )

Return failure



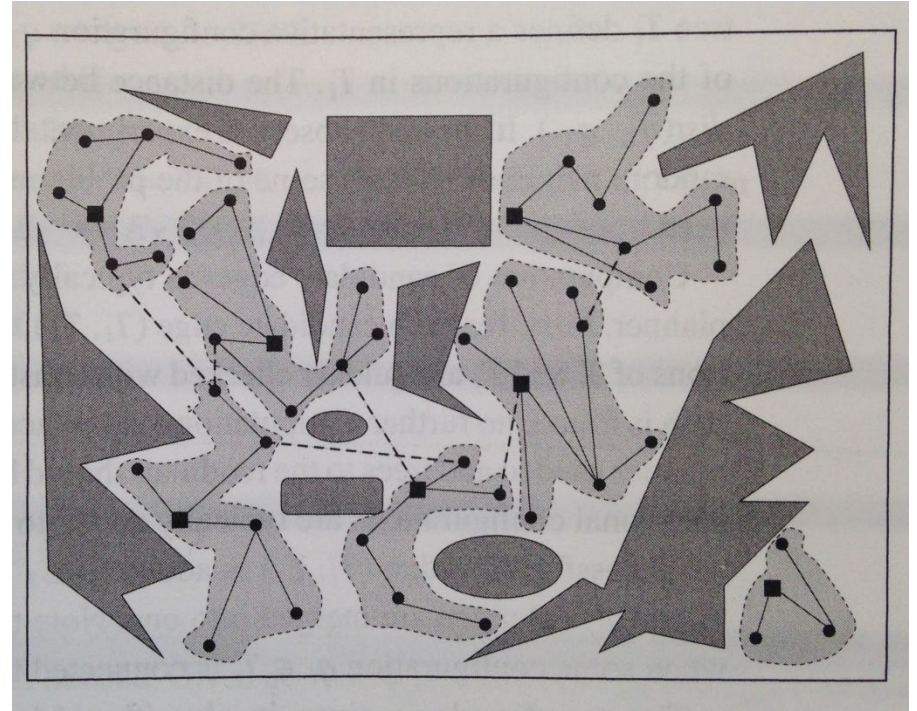
# Sampling Based Roadmap of Trees (SRT)

- Idea
  - Use a tree based planner (RRT or EST) instead of the local planner when building a PRM.
- Can be used both as single and multi query planner



# SRT

- Generate random samples.
- Grow EST or RRT trees from these.
- Try to connect the trees by local planner or by tree based planner.



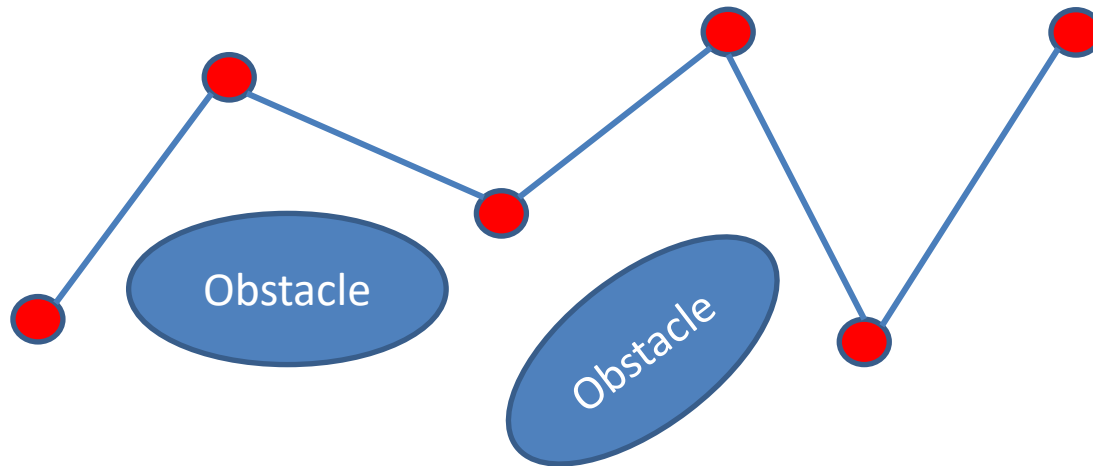
# Post Processing

- Objectives
  - Path length
  - Smoothness
  - Clearance



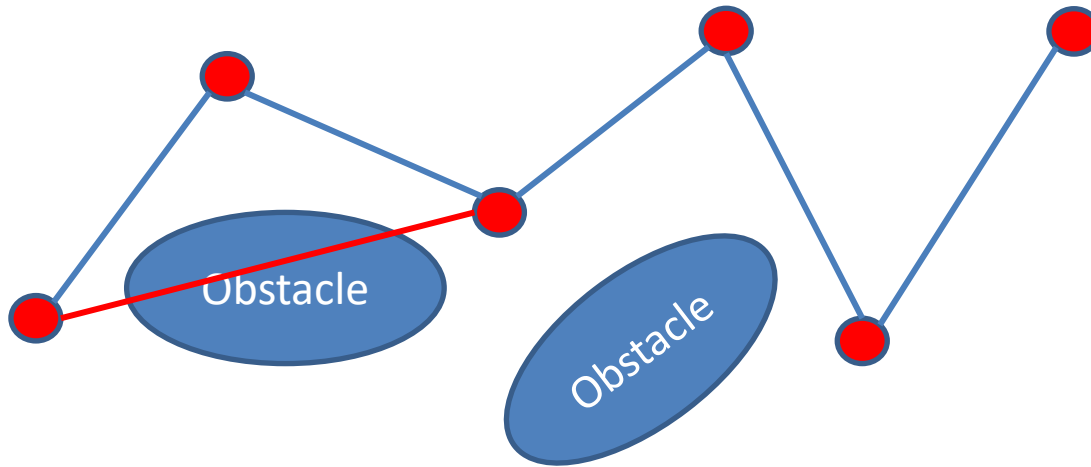
# Post Processing – Path Length

- Try to see if we can connect two neighbors



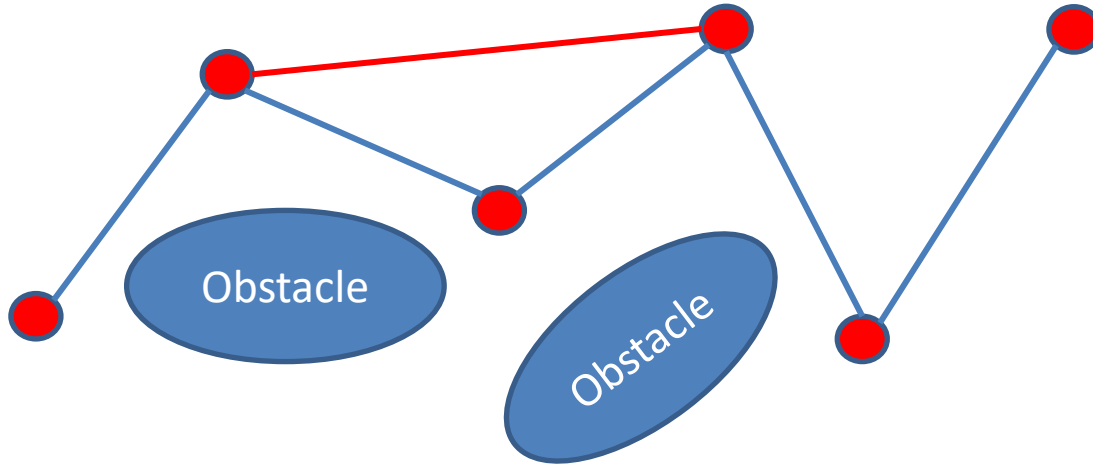
# Post Processing – Path Length

- Try to see if we can connect two neighbors



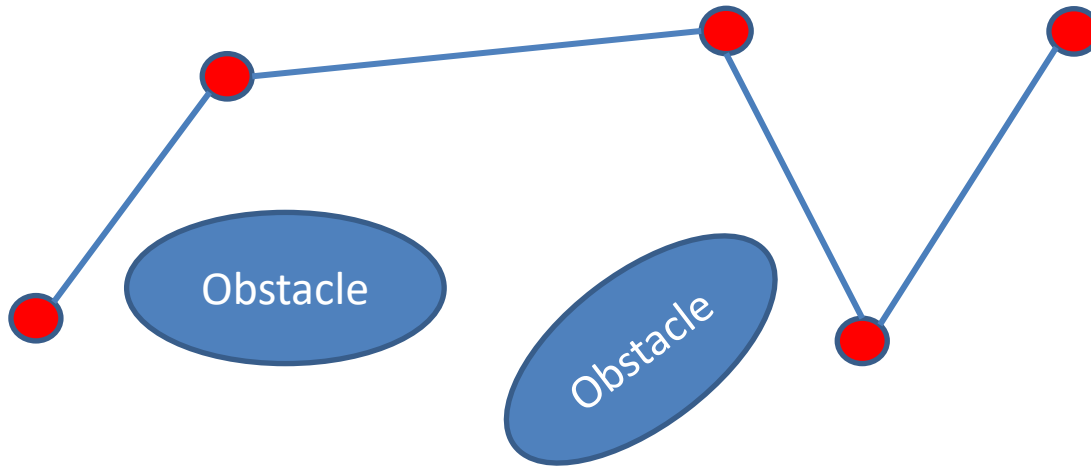
# Post Processing – Path Length

- Try to see if we can connect two neighbors



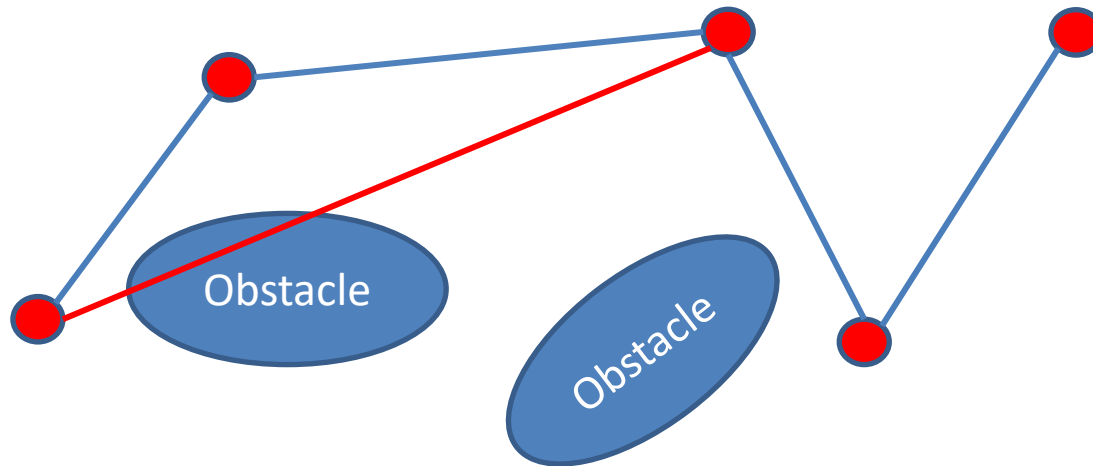
# Post Processing – Path Length

- Try to see if we can connect two neighbors



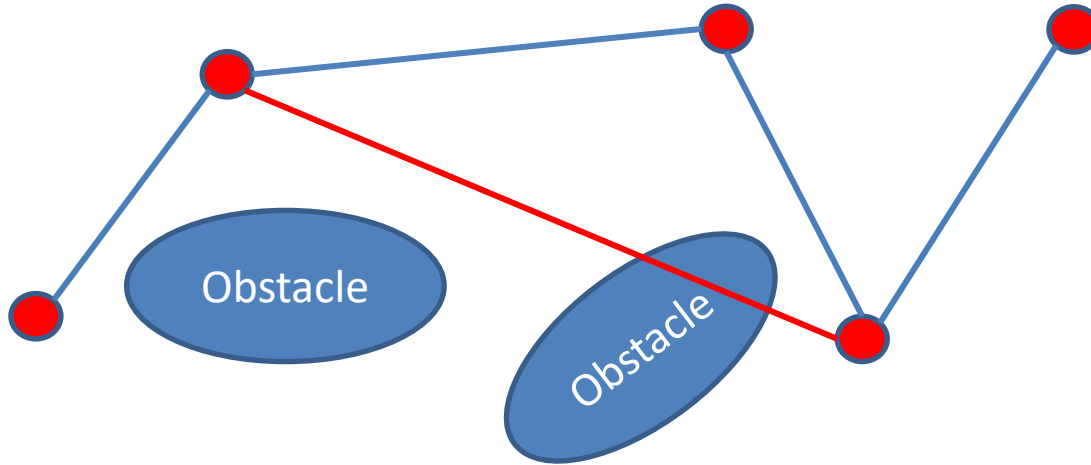
# Post Processing – Path Length

- Try to see if we can connect two neighbors



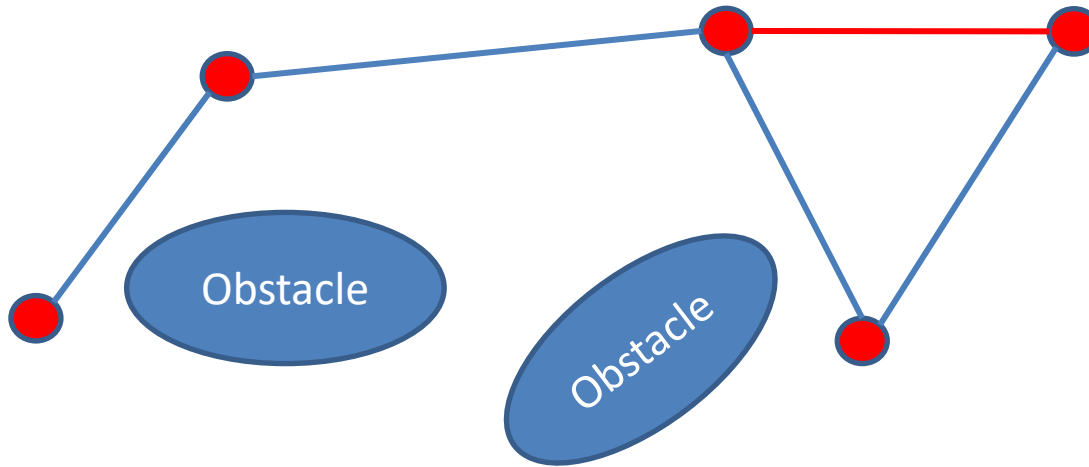
# Post Processing – Path Length

- Try to see if we can connect two neighbors



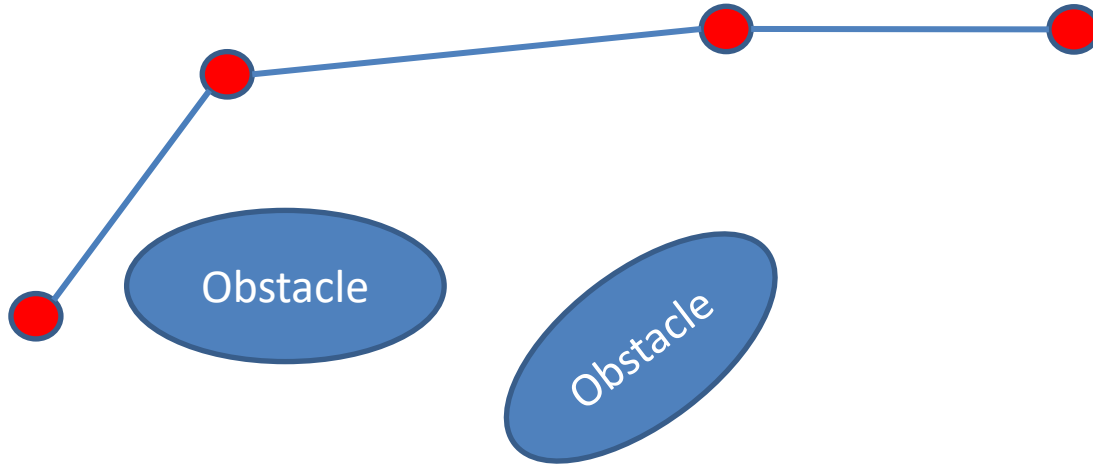
# Post Processing – Path Length

- Try to see if we can connect two neighbors



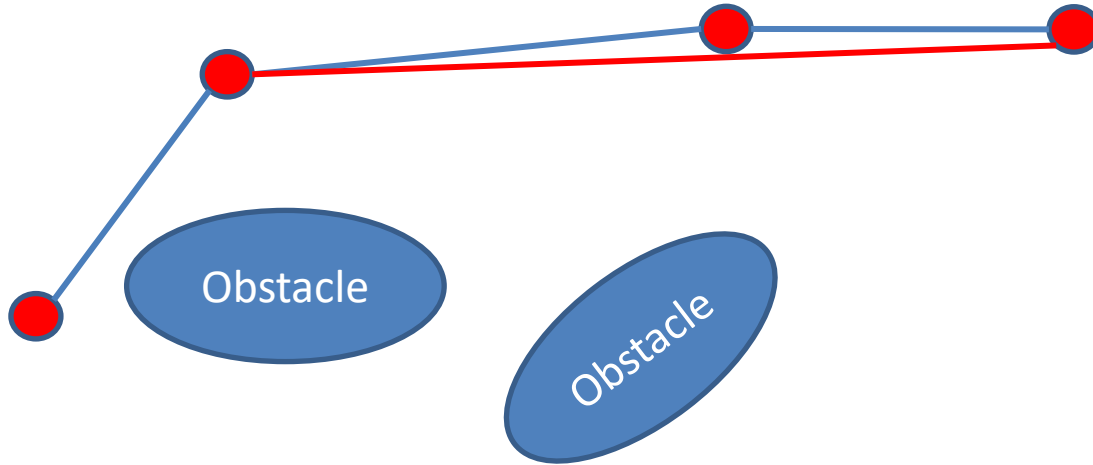
# Post Processing – Path Length

- Try to see if we can connect two neighbors



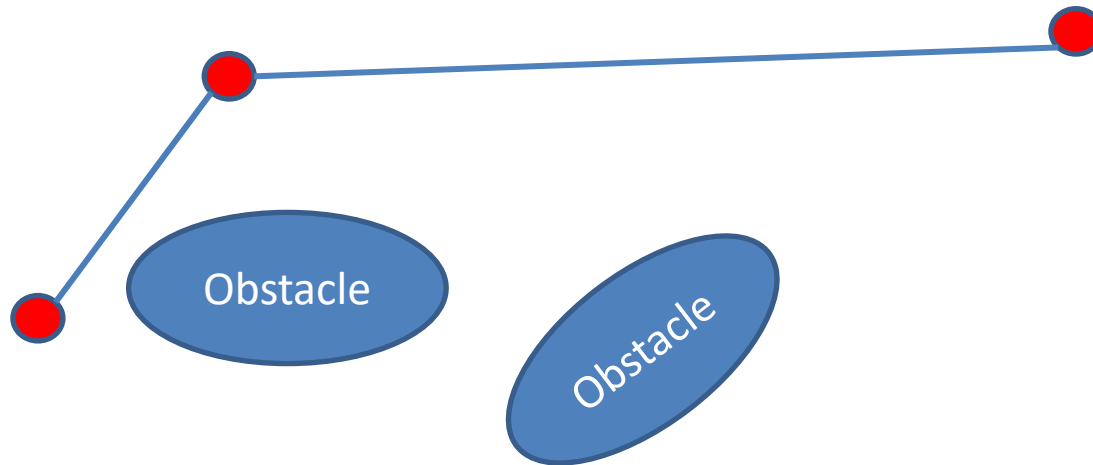
# Post Processing – Path Length

- Try to see if we can connect two neighbors



# Post Processing – Path Length

- Try to see if we can connect two neighbors



# Summary

- Sampling based planners
  - Multiquery: PRM, SRT
  - Single query: EST, RRT, (RRT\*)
- Two different schemes:
  - Roadmap is learnt once, used for multiple queries
  - Every query is addressed from scratch
- Tuning the sampling strategy matters
  - Coverage / focused
  - Random vs. pseudo random



- Programming exercise
- We need a volunteer for **RRT\*** (next week):  
—



# Midterm evaluation

